2026 IEEE 23rd International Conference on Software Architecture Companion (ICSA-C) | 979-8-3315-7787-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICSA-C68850.2026.00030

Mining Architectural Decisions and Trade-Offs from ADRs with LLMs at Scale

Ruoyu Su*, Matteo Esposito*, Davide Taibi†, Valentina Lenarduzzi†
*University of Oulu, Finland †University of Southern Denmark, Vejle, Denmark
{ruoyu.su, matteo.esposito}@oulu.fi {taibi, lenarduzzi}@imada.sdu.dk

***Abstract*—Architectural decisions are central to software architecture practice, yet their rationale is often fragmented across heterogeneous artifacts such as Architecture Decision Records (ADRs), pull request discussions, code review comments, and issue threads. Large Language Models (LLMs) provide new opportunities to mine architectural decisions and trade-offs from these unstructured sources. However, most existing approaches remain interactive and difficult to scale, limiting reproducibility, cost control, and methodological rigor when analyzing decision-making across large software portfolios. This short tutorial presents practical lessons learned from running LLM-based architectural decision mining at scale through High-Performance Computing (HPC) batch inference. We focus on two widely adopted technologies: (i) SLURM batch scheduling (`sbatch`) for deterministic and resource-aware execution, and (ii) `vLLM` for high-throughput GPU inference on shared infrastructures. The tutorial demonstrates how batch inference enables architecture researchers and practitioners to operationalize decision mining as a repeatable pipeline, with explicit input schemas, prompt versioning, structured outputs, and systematic validation. We further discuss how to treat LLM outputs as judgments rather than ground truth, combining multiple models and targeted human sampling to ensure the trustworthiness of extracted architectural knowledge.**

***Index Terms*—Architectural degradation, Architectural Decision Record, Architectural Design Decisions, Large Language Models**

I. Description of the Topic

Architectural Decision Mining aims to extract and structure decision-related knowledge from development artifacts, including decisions, alternatives, trade-offs, rationale, and consequences [1]. In practice, such knowledge is rarely captured in a single location. While ADRs provide an explicit format, many decisions remain implicit in pull request discussions, code review comments, and issue negotiations, especially when teams discuss quality attribute trade-offs such as performance, maintainability, security, and scalability [2].

LLMs are increasingly used to support decision mining tasks such as (i) identifying whether a discussion contains an architectural decision, (ii) extracting the decision statement and rationale, (iii) linking decisions to alternatives and consequences, and (iv) summarizing trade-offs across multiple discussions. However, decision mining at scale introduces challenges in terms of reproducibility, execution cost, and uncontrolled variability across runs [3].

This tutorial addresses a central challenge for the software architecture community: *how to execute LLM-based architectural decision mining at repository and portfolio scale without sacrificing empirical rigor*. We present a batch-oriented execution model based on HPC infrastructure, enabling repeatable, controlled experiments across thousands of ADRs and pull request threads [4].

II. Relevance to the Software Architecture Community

Architectural decisions and their rationale represent a key form of architectural knowledge, supporting maintainability, onboarding, governance, and long-term evolution [1]. However, decision documentation is often incomplete, outdated, or scattered. Therefore, methods that support scalable extraction of architectural decisions from real-world artifacts are increasingly important for both research and industry.

At the same time, the software architecture community requires methodologies that ensure transparency and trustworthiness when applying LLMs to architecture tasks. Interactive usage makes it difficult to replicate results, quantify uncertainty, or compare approaches across datasets and systems. Hence, HPC-based batch inference is not merely an implementation detail but an enabler for rigorous decision-mining pipelines: it supports deterministic execution, explicit resource allocation, prompt versioning, structured outputs, and systematic validation strategies.

III. Format and Intended Audience

We propose a **Short Tutorial (90 minutes)**. The tutorial targets: (i) software architecture researchers working on architectural knowledge, rationale, and evolution; (ii) practitioners interested in extracting and maintaining architectural decision knowledge across teams; (iii) PhD students seeking scalable, reproducible methods for LLM-based architecture analytics.

IV. Outline (90 Minutes)

Part 1 (20 min): Architectural Decision Mining at Scale
Goal: align on what "decision mining" means in practice and why scale changes the game.

- **Decision artifacts and signals:** ADR templates, PR threads, code review comments, issue discussions, commit messages as supporting evidence.
- **What to extract (target schema):** decision statement, status (proposed/accepted/superseded), context, alternatives, rationale, consequences, stakeholders, and links to quality attributes.

DOI 10.1109/ICSA-C68850.2026.00030

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 14,2026 at 08:43:46 UTC from IEEE Xplore. Restrictions apply.

- **Trade-offs as first-class outputs:** examples such as performance vs. maintainability, security vs. usability, latency vs. cost.
- **Challenges at scale:** fragmented rationale, implicit decisions, long threads, noisy discussions, inconsistent terminology, and cross-artifact traceability.
- **Running example setup:** a small bundle of ADRs + associated PR discussions; define the desired JSON output and what "success" looks like.

Part 2 (25 min): Reproducible Batch Execution with SLURM

Goal: turn decision mining into a controlled pipeline rather than ad-hoc prompting.

- **Pipeline decomposition:** (1) artifact collection and preprocessing, (2) inference jobs, (3) post-processing and aggregation, (4) validation sampling.
- **SLURM patterns for scale:** job arrays for thousands of ADR/PR items, resource requests (GPU/CPU/memory), time limits, and queue selection.
- **Determinism and traceability:** fixed seeds (where applicable), pinned model versions, pinned prompt versions, immutable input manifests, and run metadata.
- **Logging and observability:** stdout/stderr capture, per-item logs, structured run summaries, and failure diagnostics.
- **Robustness:** retries, checkpointing, partial reruns, and safe handling of malformed outputs.
- **Cost-aware execution:** throughput vs. latency trade-offs, selecting model sizes, batching strategy implications for GPU hours.

Part 3 (25 min): Efficient Inference with `vLLM`

Goal: show how to serve local LLMs efficiently and extract structured decision knowledge.

- **Why `vLLM` for batch inference:** high throughput, efficient KV-cache management, serving on shared GPUs.
- **Serving configuration essentials:** model loading, tensor parallelism (if applicable), max tokens, concurrency, and batching controls.
- **Prompt design for decision mining:** role prompts, extraction instructions, and enforcing consistent output fields across ADRs and PR threads.
- **Structured outputs:** JSON schemas for decision elements; strategies to reduce formatting errors (constraints, delimiters, repair prompts).
- **Post-processing:** schema validation, JSON repair, normalization (labels for decision types and quality attributes), and linking ADRs to PR evidence.
- **Hands-on mini walk-through:** one ADR + one PR thread into the pipeline, then show the aggregated decision record.

Part 4 (15 min): Validation and Trustworthiness

Goal: treat LLM outputs as judgments and make uncertainty explicit.

- **Judgment framing:** why extracted decisions are hypotheses requiring validation, especially for implicit decisions.
- **Triangulation:** multi-model agreement (e.g., two different LLM families) and adjudication strategies.
- **Disagreement analysis:** what disagreement means (ambiguity vs. model failure), and how to prioritize items for review.
- **Human-in-the-loop sampling:** targeted sampling for high-risk cases (security decisions, high-impact trade-offs), stratification by artifact type.
- **Quality controls:** precision-oriented checks (false decision detection), completeness checks (missing alternatives/rationale), and drift checks across time.

Part 5 (5 min): Takeaways and Discussion

Goal: leave attendees with an actionable checklist.

- **Checklist:** inputs/manifests, prompt and model pinning, SLURM execution pattern, schema validation, and validation plan.
- **Open challenges:** implicit decisions, cross-thread linking, long-context limits, evolving terminology, and dataset release considerations.
- **Q&A and next steps:** how attendees can adapt the pipeline to their own systems and research questions.

V. Speaker Information and Biographies

Ruoyu Su is a doctoral researcher at the University of Oulu. His work focuses on LLM in Software Architecture Quality and Maintenance.

Matteo Esposito is a postdoctoral researcher at the University of Oulu. His research interests include software architecture, secure software engineering, and the application of LLMs to software quality and architectural knowledge extraction.

Davide Taibi is a full professor at the University of Southern Denmark. His research focuses on software architecture, architectural technical debt, and empirical studies on software quality and maintainability.

Valentina Lenarduzzi is a fullprofessor at the University of Southern Denmark. Her research interests include software architecture, software quality, and AI techniques for software engineering and architecture analysis.

References

[1] A. Shahbazian, Y. K. Lee, D. Le, Y. Brun, and N. Medvidovic, "Recovering architectural design decisions," in *2018 IEEE International Conference on Software Architecture (ICSA)*. IEEE, 2018, pp. 95–9509.

[2] J. Tyree and A. Akerman, "Architecture decisions: Demystifying architecture," *IEEE software*, vol. 22, no. 2, pp. 19–27, 2005.

[3] V. De Martino, J. Castaño, F. Palomba, X. Franch, and S. Martínez-Fernández, "A methodological framework for llm-based mining of software repositories," *arXiv preprint arXiv:2508.02233*, 2025.

[4] R. Su, A. Bakhtin, N. Ahmad, M. Esposito, V. Lenarduzzi, and D. Taibi, "Evaluating large language models for detecting architectural decision violations," in *Proceedings of the 23rd IEEE International Conference on Software Architecture (ICSA 2026)*, Amsterdam, The Netherlands, Jun. 2026, accepted on 08.02.2026. [Online]. Available: https://arxiv.org/abs/2602.07609

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 14,2026 at 08:43:46 UTC from IEEE Xplore. Restrictions apply.